Python + pytest Essentials

```
# region START HERE: pitch, how to run, index
"""
═══════════════════════════════════════════════════════════════════════════════
PYTHON + PYTEST ESSENTIALS: the concepts interviewers ask about, explained through THIS
framework (mirrored in Python), with model answers. Every claim below is a RUNNING TEST.
═══════════════════════════════════════════════════════════════════════════════

FOLD / UNFOLD (VS Code): every section is a `# region`.
  Ctrl+K Ctrl+0  fold all → only section titles      Ctrl+K Ctrl+J  unfold all
  Ctrl+G + a line number from the INDEX below jumps there.

HOW TO RUN (from the project root)
  py -3.15 -m venv notes/python/.venv                                  # once
  notes/python/.venv/Scripts/python -m pip install -r notes/python/requirements.txt
  notes/python/.venv/Scripts/python -m pytest notes/python             # everything
  notes/python/.venv/Scripts/python -m pytest notes/python -m smoke    # just smoke
  notes/python/.venv/Scripts/python -m pytest notes/python --env prod  # data tests skip
  notes/python/.venv/Scripts/python -m pytest notes/python --setup-show -k end_to_end
                                                 # ↑ prints every fixture SETUP/TEARDOWN

THE FILES (the same 4 layers as the TypeScript framework)
  test_python_pytest_essentials.py   LAYER 1  WHAT    specs + these notes
  conftest.py                        LAYER 2  WIRING  fixtures (DI) + hooks
  framework.py                       LAYER 3  HOW     ApiClient, services, assertions
                                     LAYER 4  WHERE   environments, pydantic schemas, builders
  search_cases.json                  data for the data-driven example (section 5)
  pytest.ini                         markers, --strict-markers, xfail_strict

┌─────────────────────────────────────────────────────────────────────────────┐
│ THE 2-MINUTE PITCH: "What's your Python / pytest experience?"               │
└─────────────────────────────────────────────────────────────────────────────┘
 "My current framework is Playwright with TypeScript, but its design maps one-to-one onto
  Python and pytest, and I can build it either way.

  Specs only describe behaviour. pytest fixtures in conftest.py are the dependency injection:
  a test asks for `auth_api` and `registered_user` in its signature, pytest builds them, and a
  yield fixture deletes the account afterwards even if the test fails. Expensive things like
  the HTTP session are session-scoped. Page objects and API service classes own selectors and
  HTTP, an environment strategy is chosen with a --env option I add in pytest_addoption, and
  a collection hook skips data-changing tests on prod.

  API responses are validated with pydantic, the Python equivalent of zod: one model gives the
  runtime check and the type hints. Tests are data-driven with parametrize, selected with
  registered markers like smoke and regression, run in parallel with pytest-xdist, and report
  to JUnit XML for Jenkins and to Allure. For the browser I'd use pytest-playwright, whose
  fixtures (page, context, browser) are the same model as Playwright Test's."

┌─────────────────────────────────────────────────────────────────────────────┐
│ INDEX                                                                       │
└─────────────────────────────────────────────────────────────────────────────┘
 1. PYTHON ESSENTIALS FOR AUTOMATION ..................................... line 117
    Hook      The 8 language traps behind most "simple" Python interview questions.
    Summary   mutable defaults · is vs == · copies · comprehensions/generators · *args ·
              decorators (retry) · context managers · exceptions · GIL.
 2. TYPE HINTS + GENERICS (the TypeScript bridge) ........................ line 316
    Hook      Hints are documentation a checker reads; Python ignores them at runtime.
    Summary   TypeVar = <T> · Protocol = TS interface · Literal · TypedDict · mypy/pyright.
 3. LAYERED ARCHITECTURE IN A PYTEST PROJECT ............................. line 383
    Hook      Tests say WHAT, pages/services know HOW, config knows WHERE.
 4. FIXTURES = DEPENDENCY INJECTION ...................................... line 430
    Hook      Ask for it by name in the signature; `yield` splits setup from teardown.
    Summary   5 scopes · yield teardown · factory fixtures · autouse · overrides · built-ins.
 5. PARAMETRIZE + DATA-DRIVEN TESTS ...................................... line 565
    Hook      One test body, many rows; ids make each row readable in reports.
 6. MARKERS, SELECTION, SKIP AND XFAIL ................................... line 636
    Hook      Markers are tags; -m selects by tag, -k by name.
 7. HOOKS, PLUGINS AND CONFIGURATION ..................................... line 698
    Hook      conftest.py hooks change pytest itself; plugins are packaged conftests.
 8. RUNTIME VALIDATION WITH PYDANTIC (= zod) ............................. line 780
 9. DESIGN PATTERNS IN PYTHON ............................................ line 850
10. PLAYWRIGHT FOR PYTHON (pytest-playwright) ............................. line 935
11. PARALLEL RUNS, FLAKINESS AND CI ...................................... line 996
12. ONE TEST, END TO END ................................................. line 1062
13. TYPESCRIPT → PYTHON CHEAT SHEET ...................................... line 1105
    TEST YOURSELF (20 questions) ........................................ line 1145

HOW EACH SECTION IS BUILT
  HOOK       one line to remember
  WHY        why a lead/architect cares
```

```
   CODE       runnable, with the TypeScript equivalent named
   PROOF      test_* functions: if a claim in these notes were wrong, pytest would fail
   INTERVIEW  question → model answer → follow-ups → trade-off → mistake to avoid
"""
# endregion

from __future__ import annotations

import copy
import functools
import json
import os
import time
from collections.abc import Callable, Iterator
from contextlib import contextmanager
from pathlib import Path
from typing import Any, Protocol, TypedDict, TypeVar

import pytest
from pydantic import BaseModel, Field, field_validator

from framework import (
    AuthApi,
    EnvironmentConfig,
    FakeAutomationExerciseServer,
    NewUser,
    Product,
    ProductsApi,
    ProductsList,
    UserBuilder,
    build_user,
    expect_api_code,
    expect_schema,
    get_environment,
)


# region 1. PYTHON ESSENTIALS FOR AUTOMATION
"""
═══════════════════════════════════════════════════════════════════════════════
 1. PYTHON ESSENTIALS FOR AUTOMATION
═══════════════════════════════════════════════════════════════════════════════
 HOOK  The 8 language traps behind most "simple" Python interview questions.
 WHY   A framework is shared code: one of these bugs in a helper breaks hundreds of tests,
       usually only in CI, usually only sometimes.
"""


# ── 1.1 MUTABLE DEFAULT ARGUMENTS ────────────────────────────────────────────
# The default is created ONCE, when `def` runs, and then shared by every call.
def add_tag_buggy(tag: str, tags: list[str] = []) -> list[str]:  # noqa: B006 (the bug on purpose)
    tags.append(tag)
    return tags


def add_tag_fixed(tag: str, tags: list[str] | None = None) -> list[str]:
    tags = [] if tags is None else tags  # a NEW list per call
    tags.append(tag)
    return tags


def test_mutable_default_is_shared_between_calls() -> None:
    assert add_tag_buggy("smoke") == ["smoke"]
    assert add_tag_buggy("api") == ["smoke", "api"]  # ← leaked from the previous call!
    assert add_tag_fixed("smoke") == ["smoke"]
    assert add_tag_fixed("api") == ["api"]  # independent
    # Same trap in dataclasses: use field(default_factory=dict) (see framework.UserBuilder).


# ── 1.2 `is` vs `==` ─────────────────────────────────────────────────────────
def test_is_checks_identity_equals_checks_value() -> None:
    a, b = [1, 2], [1, 2]
    assert a == b  # same VALUE
    assert a is not b  # different OBJECTS
    result = None
    assert result is None  # `is None` is the idiom: None is a singleton
    # Never `x is 5` or `name is "qa"`: it may work by accident (interning) and then not.


# ── 1.3 SHALLOW vs DEEP COPY ─────────────────────────────────────────────────
def test_shallow_copy_shares_nested_objects() -> None:
```

```
    base_payload = {"user": {"name": "QA"}, "tags": ["smoke"]}
    shallow = dict(base_payload)  # also: base_payload.copy(), {**base_payload}
    deep = copy.deepcopy(base_payload)
    shallow["user"]["name"] = "changed"
    assert base_payload["user"]["name"] == "changed"  # ← the shared inner dict changed too
    assert deep["user"]["name"] == "QA"  # deepcopy is fully independent
    # TS equivalent: spread {...obj} is shallow, structuredClone(obj) is deep.


# ── 1.4 COMPREHENSIONS AND GENERATORS ──────────────────────────────────────
def test_comprehensions_and_lazy_generators() -> None:
    products = [{"name": "Blue Top", "price": 500}, {"name": "Dress", "price": 1000}]
    names = [p["name"] for p in products if p["price"] < 800]  # list comprehension: filter+map
    by_name = {p["name"]: p["price"] for p in products}  # dict comprehension
    assert names == ["Blue Top"]
    assert by_name["Dress"] == 1000

    calls: list[int] = []

    def numbers() -> Iterator[int]:  # a GENERATOR function: `yield` makes it lazy
        for n in range(1_000_000):
            calls.append(n)
            yield n

    first_even_over_10 = next(n for n in numbers() if n > 10 and n % 2 == 0)
    assert first_even_over_10 == 12
    assert len(calls) == 13  # produced only what was needed, not a million items
    # The same `yield` is what powers pytest fixtures: code before yield = setup, after = teardown.


# ── 1.5 *args AND **kwargs ─────────────────────────────────────────────────
def test_args_and_kwargs_collect_and_spread() -> None:
    def request(method: str, *path_parts: str, **query: str) -> str:
        url = "/".join(path_parts)
        qs = "&".join(f"{k}={v}" for k, v in query.items())
        return f"{method} /{url}?{qs}"

    assert request("GET", "api", "products", page="1", size="10") == "GET /api/products?page=1&size=10"
    parts, params = ("api", "brands"), {"page": "2"}
    assert request("GET", *parts, **params) == "GET /api/brands?page=2"  # * and ** also SPREAD


# ── 1.6 DECORATORS (a retry helper, the classic interview task) ────────────
def retry(times: int, delay_s: float = 0.0, exceptions: tuple[type[BaseException], ...] = (AssertionError,)):
    """Decorator FACTORY: retry(3) returns the real decorator. Use for polling helpers, NOT to
    hide flaky tests (prefer fixing the cause, or pytest-rerunfailures with a visible report)."""

    def decorator(func: Callable[..., Any]) -> Callable[..., Any]:
        @functools.wraps(func)  # keeps func.__name__ / docstring (reports, debugging)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions:
                    if attempt == times:
                        raise  # last attempt: re-raise the ORIGINAL error with its traceback
                    time.sleep(delay_s)
            raise AssertionError("unreachable")

        return wrapper

    return decorator


def test_retry_decorator_retries_then_succeeds() -> None:
    attempts = {"count": 0}

    @retry(times=3)
    def eventually_ready() -> str:
        attempts["count"] += 1
        assert attempts["count"] >= 3, "not ready yet"
        return "ready"

    assert eventually_ready() == "ready"
    assert attempts["count"] == 3
    assert eventually_ready.__name__ == "eventually_ready"  # thanks to functools.wraps


def test_retry_decorator_reraises_after_last_attempt() -> None:
    @retry(times=2)
    def never_ready() -> None:
```

```
        raise AssertionError("still down")

    with pytest.raises(AssertionError, match="still down"):
        never_ready()


# ── 1.7 CONTEXT MANAGERS (`with`) ──────────────────────────────────────────
@contextmanager
def temporary_env(name: str, value: str) -> Iterator[None]:
    """Set an env var for a block and ALWAYS restore it. (In tests prefer monkeypatch, sec. 4.)"""
    old = os.environ.get(name)
    os.environ[name] = value
    try:
        yield  # the body of the `with` runs here
    finally:  # runs even if the body raised: that's the whole point
        if old is None:
            os.environ.pop(name, None)
        else:
            os.environ[name] = old


def test_context_manager_restores_even_on_error() -> None:
    os.environ.pop("DEMO_FLAG", None)
    with pytest.raises(RuntimeError):
        with temporary_env("DEMO_FLAG", "on"):
            assert os.environ["DEMO_FLAG"] == "on"
            raise RuntimeError("boom")
    assert "DEMO_FLAG" not in os.environ  # restored anyway
    # Real uses: `with sync_playwright() as p:`, `with open(...)`, `with requests.Session() as s:`


# ── 1.8 EXCEPTIONS: custom types and `raise ... from` ──────────────────────
class EnvironmentError_(Exception):
    """A domain-specific error type makes failures searchable and catchable by meaning."""


def test_raise_from_keeps_the_cause() -> None:
    with pytest.raises(EnvironmentError_) as info:
        try:
            {"qa": 1}["prd"]
        except KeyError as error:
            raise EnvironmentError_("TEST_ENV=prd is not a known environment") from error
    assert isinstance(info.value.__cause__, KeyError)  # the original error is chained, not lost
    # framework.get_environment uses `from None` instead: it deliberately HIDES the KeyError
    # because the new message already says everything.


# ── 1.9 THE GIL, THREADS, PROCESSES, ASYNC ─────────────────────────────────
"""
 • The standard CPython build has a GIL: one thread runs Python bytecode at a time.
   Threads still help for I/O (HTTP calls wait on the network, releasing the GIL).
   Free-threaded (no-GIL) builds exist since 3.13 but are opt-in.
 • pytest-xdist uses PROCESSES (-n 4 = 4 worker processes), so CPU and the GIL don't matter,
   and each worker has its OWN memory: session fixtures are per worker (section 11).
 • async/await: one thread, many waits interleaved. Playwright for Python has a sync API
   (default in pytest-playwright) and an async API; don't mix them in one test.

 INTERVIEW
  Q  "What happens with a mutable default argument?"
  A  "The default is evaluated once when the function is defined, so every call shares the same
      list or dict. In a test helper that leaks data between tests. Use None and create the
      object inside, or field(default_factory=...) in a dataclass."
  Q  "Shallow vs deep copy, and why does it matter for test data?"
  A  "A shallow copy duplicates only the outer container; nested dicts are shared, so changing
      one test's payload changes the template. I build fresh data per test with a builder, or
      deepcopy a template."
  Q  "Write a retry decorator."  → retry() above: decorator factory, functools.wraps, re-raise
      the original on the last attempt, and only catch the exceptions you expect.
  Mistake  Catching bare `except:` or `except Exception: pass` in helpers: it swallows
           AssertionError and KeyboardInterrupt-like signals and makes failures invisible.
"""
# endregion


# region 2. TYPE HINTS + GENERICS
"""
═══════════════════════════════════════════════════════════════════════════
 2. TYPE HINTS + GENERICS (the TypeScript bridge)
═══════════════════════════════════════════════════════════════════════════
 HOOK  Hints are documentation that a checker (mypy / pyright) reads. Python itself IGNORES
       them at runtime, which is why API responses still need pydantic (section 8).
```

```
 TS → Python
   string | undefined           →  str | None              (Optional[str] in old code)
   'admin' | 'user'             →  Literal["admin", "user"]
   interface Named { name }     →  class Named(Protocol): name: str      (structural typing)
   type Row = { id: number }    →  class Row(TypedDict): id: int        (shape of a dict)
   function f<T>(x: T): T       →  T = TypeVar("T"); def f(x: T) -> T   (3.12+: def f[T](x: T))
   <T extends { id: number }>   →  TypeVar("T", bound=HasId)
   readonly / as const          →  @dataclass(frozen=True), Final, tuple instead of list
"""


class HasId(Protocol):  # ANY object with an int `id` matches; no inheritance needed (duck typing)
    id: int


T = TypeVar("T", bound=HasId)  # TS: <T extends { id: number }>


def index_by_id(items: list[T]) -> dict[int, T]:
    return {item.id: item for item in items}  # keeps the full type T for the caller


class ProductRow(TypedDict):
    id: int
    name: str


def test_generics_and_protocols_keep_the_concrete_type() -> None:
    products = [Product(id=1, name="Blue Top", price="Rs. 500")]
    by_id = index_by_id(products)  # T = Product → the checker knows by_id[1].price exists
    assert by_id[1].price == "Rs. 500"

    row: ProductRow = {"id": 7, "name": "Dress"}
    assert row["name"] == "Dress"


def test_type_hints_are_not_enforced_at_runtime() -> None:
    def total(price: int, qty: int) -> int:
        return price * qty

    assert total("Rs. ", 2) == "Rs. Rs. "  # type: ignore[arg-type]  ← Python runs it anyway!
    # → hints catch bugs in YOUR code (via mypy/pyright in CI); data from OUTSIDE (API bodies,
    #   JSON files, env vars) must be validated at runtime: pydantic.


"""
 INTERVIEW
  Q  "Python is dynamically typed. Do you use type hints in a test framework?"
  A  "Yes, everywhere in the framework layer: page objects, services and fixtures. They give
      autocomplete in tests and let mypy or pyright catch a wrong argument in CI before a test
      run. They're not checked at runtime, so for API responses I validate with pydantic."
  Q  "What's a Protocol?"
  A  "Structural typing, like a TypeScript interface: anything with the right attributes
      matches, without inheriting. Useful for fakes: a FakeClient just needs a send() method."
  Trade-off  Strict typing on test bodies adds noise; I type the framework strictly, tests lightly.
"""
# endregion


# region 3. LAYERED ARCHITECTURE IN A PYTEST PROJECT
"""
═══════════════════════════════════════════════════════════════════════════════════════
 3. LAYERED ARCHITECTURE IN A PYTEST PROJECT
═══════════════════════════════════════════════════════════════════════════════════════
 HOOK  "Tests say WHAT, pages/services know HOW, config knows WHERE." (same as TypeScript)

 THE TYPESCRIPT FRAMEWORK, REBUILT IN PYTHON
   parabank-py/
   ├── pyproject.toml            deps + [tool.pytest.ini_options] (markers, addopts, testpaths)
   ├── conftest.py               ROOT fixtures: env_config, api session, logged_in page
   ├── config/
   │   ├── environments.py       ENVIRONMENTS dict (strategy) + get_environment()
   │   └── settings.py           secrets from .env (python-dotenv / pydantic-settings)
   ├── pages/                    LoginPage, TransferFundsPage   (page objects)
   ├── components/               AccountServicesMenu             (component objects)
   ├── api/
   │   ├── client.py             ApiClient: the ONE requests.Session
   │   ├── services/             ProductsApi, AccountApi, AuthApi (facades)
   │   ├── schemas.py            pydantic models
   │   └── assertions.py         expect_schema, expect_api_code (adapter)
```

```
   ├── test_data/               builders + JSON/CSV data files
   └── tests/
       ├── ui/conftest.py       UI-only fixtures (browser_context_args, storage_state)
       ├── ui/test_login.py
       └── api/conftest.py      API-only fixtures (registered_user)
           api/test_verify_login.py

 conftest.py HIERARCHY = fixture visibility
   root conftest.py       → visible to ALL tests
   tests/api/conftest.py  → visible only under tests/api, can OVERRIDE a root fixture by name
   No imports needed: pytest discovers conftest files and injects fixtures by parameter name.

 IN THIS FOLDER: section 12 traces one test through all 4 layers.

 INTERVIEW
  Q  "How would you structure a pytest framework?"
  A  "Four layers with one-way dependencies: tests describe behaviour; conftest.py fixtures wire
      dependencies and own setup and cleanup; page objects and API services own selectors and
      HTTP; config owns environments, secrets and data. A conftest per area keeps UI fixtures out
      of API tests. Lower layers never import tests or conftest."
  Mistake  Importing fixtures from conftest.py (`from conftest import ...`): fixtures are
           injected by name; importing breaks discovery and the conftest hierarchy.
"""
# endregion


# region 4. FIXTURES = DEPENDENCY INJECTION
"""
═══════════════════════════════════════════════════════════════════════════════
 4. FIXTURES = DEPENDENCY INJECTION
═══════════════════════════════════════════════════════════════════════════════
 HOOK  "Ask for it by NAME in the signature; `yield` splits setup from teardown."

 WHY   Setup and cleanup in ONE place; cleanup runs even when the test fails; tests get only
       what they ask for (lazy); expensive things are shared with a wider scope.

 THE 5 SCOPES (TS has 2: test, worker)
   function (default)  fresh per test                    ← TS test scope
   class               once per test class
   module              once per test FILE
   package             once per package (folder with __init__.py)
   session             once per run, BUT once per xdist worker in parallel ← TS worker scope
   Rule: a fixture may only depend on fixtures of the SAME or WIDER scope
         (a session fixture asking for a function fixture → ScopeMismatch error).

 conftest.py IN THIS FOLDER (read it next to this section)
   env_config, ae_server            session  (built once)
   api_client, *_api                function (cheap, fresh)
   registered_user                  function, yield → create / YIELD / delete
   make_user                        factory-as-fixture: many users, all cleaned up
   _count_requests_per_test         autouse: runs for every test without being asked

 HOW IT RUNS for  def test_x(auth_api, registered_user):
   session start    ae_server built once (it's session-scoped)
   before test      api_client → auth_api; account_api → registered_user (creates account)
   test body        runs with auth_api and registered_user injected
   after test       registered_user teardown (delete) → other fixtures torn down in REVERSE order
   Fixtures nobody asked for (products_api) are never built: lazy.
   See it live:  pytest notes/python --setup-show -k end_to_end
"""


def test_session_fixture_is_shared_function_fixture_is_fresh(ae_server, api_client, products_api) -> None:
    assert products_api.client is api_client  # same test → same function-scoped instance
    assert api_client._server is ae_server  # the session-scoped server underneath


@pytest.mark.creates_data
def test_yield_fixture_cleans_up_after_the_test(registered_user: NewUser, ae_server) -> None:
    assert registered_user.email in ae_server.users  # created BEFORE the test
    # After this test returns (or fails), the teardown after `yield` deletes it.
    # Proof: test_no_users_left_behind (below) runs later and sees no leftovers.


@pytest.mark.creates_data
def test_factory_fixture_makes_several_users(make_user, auth_api: AuthApi) -> None:
    alice = make_user(name="Alice")
    bob = make_user(name="Bob")
    assert alice.email != bob.email  # builder gives unique data per call
    expect_api_code(auth_api.verify_login(bob.email, bob.password), 200, "User exists!")
```

```python
@pytest.fixture
def order_log() -> list[str]:
    return []


@pytest.fixture
def first(order_log: list[str]) -> Iterator[None]:
    order_log.append("setup first")
    yield
    order_log.append("teardown first")


@pytest.fixture
def second(first: None, order_log: list[str]) -> Iterator[None]:  # depends on `first`
    order_log.append("setup second")
    yield
    order_log.append("teardown second")  # torn down BEFORE `first` (reverse order)


def test_fixture_dependencies_set_up_in_order(second: None, order_log: list[str]) -> None:
    assert order_log == ["setup first", "setup second"]  # dependency first, then dependant


# ── Built-in fixtures worth knowing ──────────────────────────────────────────
def test_builtin_monkeypatch_and_tmp_path(monkeypatch: pytest.MonkeyPatch, tmp_path: Path) -> None:
    monkeypatch.setenv("TEST_ENV", "prod")  # undone automatically after the test
    assert get_environment(os.environ["TEST_ENV"]).allow_data_changes is False

    monkeypatch.setattr(time, "sleep", lambda _s: None)  # patch a function for this test only
    time.sleep(999)  # returns instantly

    data_file = tmp_path / "users.json"  # unique temp dir per test, kept for the last 3 runs
    data_file.write_text(json.dumps([{"name": "QA"}]))
    assert json.loads(data_file.read_text())[0]["name"] == "QA"
    # Others: request (the requesting test: request.node, request.param), pytestconfig
    # (CLI options), capsys/caplog (captured prints/logs), tmp_path_factory (session-wide).


# ── Overriding a fixture: closest definition wins (test class > module > conftest) ──────────
class TestProdSafety:
    @pytest.fixture
    def env_config(self) -> EnvironmentConfig:  # same NAME as the conftest fixture → overrides it
        return get_environment("prod")  # only for tests in this class

    def test_prod_forbids_data_changes(self, env_config: EnvironmentConfig) -> None:
        assert env_config.name == "prod"
        assert env_config.allow_data_changes is False


"""
 INTERVIEW
  Q  "How do pytest fixtures work? What are the scopes?"
  A  "Fixtures are pytest's dependency injection. A test lists what it needs as parameters;
      pytest resolves the graph, builds each fixture once per its scope, runs the test, then
      tears down in reverse order. With yield, code before yield is setup and after is teardown,
      and teardown runs even if the test fails. Scopes are function, class, module, package and
      session. I use session for expensive, read-only things like the HTTP session and function
      scope for anything a test mutates, like a registered user."
  Follow-ups
   Q "Fixtures vs setup_method / setUp?"
   A "xUnit-style setup runs for every test in the class whether needed or not, and passes data
      through self. Fixtures are lazy, composable, reusable across files via conftest, scoped,
      and keep setup and teardown together."
   Q "If setup before `yield` raises, does teardown run?"
   A "No, nothing was yielded, so the code after yield never runs. Teardowns of fixtures that
      DID finish setting up still run. Keep setup small, or register cleanup early with
      request.addfinalizer right after creating each resource."
   Q "How do you share one thing across ALL xdist workers?"
   A "Session scope is per worker. For truly once-per-run setup, use tmp_path_factory's shared
      base temp dir with a FileLock: the first worker creates it, others read the result."
   Q "One test needs three users. Three fixtures?"
   A "No, a factory fixture: it yields a function that creates users and records them, and the
      teardown deletes everything it created (make_user in conftest.py)."
  Trade-off  Deep fixture chains hide what a test depends on; --setup-show makes it visible.
  Mistake    Cleanup at the end of the test body: an earlier failing assert skips it.
"""
# endregion
```

```
# region 5. PARAMETRIZE + DATA-DRIVEN TESTS
"""
================================================================================
 5. PARAMETRIZE + DATA-DRIVEN TESTS
================================================================================
 HOOK  One test body, many rows; `ids` make each row readable in reports and in -k.
 TS    test-data/*.json + `for (const row of rows) test(row.name, ...)` in the TS framework.
"""


@pytest.mark.parametrize(
    ("term", "expected_count"),
    [
        ("top", 1),
        ("TOP", 1),  # case-insensitive
        ("", 3),  # empty term matches everything
        pytest.param("zzz", 0, id="unknown-term"),  # pytest.param: custom id and/or marks per row
    ],
)
def test_search_returns_expected_count(products_api: ProductsApi, term: str, expected_count: int) -> None:
    body = expect_schema(ProductsList, products_api.search(term).body)
    assert len(body.products) == expected_count
    # Report names: test_search_returns_expected_count[top-1], ...[unknown-term]


# Data from a FILE (keep data out of code; testers can add rows without touching Python)
SEARCH_CASES = json.loads(Path(__file__).with_name("search_cases.json").read_text())


@pytest.mark.parametrize("case", SEARCH_CASES, ids=[c["id"] for c in SEARCH_CASES])
def test_search_cases_from_json(products_api: ProductsApi, case: dict[str, Any]) -> None:
    body = expect_schema(ProductsList, products_api.search(case["term"]).body)
    assert [p.name for p in body.products] == case["expected_names"]


# Stacking decorators = cartesian product (2 x 2 = 4 tests)
@pytest.mark.parametrize("env_name", ["qa", "staging"])
@pytest.mark.parametrize("field_name", ["base_url", "workers"])
def test_every_env_defines_every_field(env_name: str, field_name: str) -> None:
    assert getattr(get_environment(env_name), field_name)


# INDIRECT parametrize: the row goes to a FIXTURE (request.param), not straight to the test
@pytest.fixture
def user_in_city(request: pytest.FixtureRequest, make_user) -> NewUser:
    return make_user(city=request.param)  # each row creates (and later deletes) a real user


@pytest.mark.creates_data
@pytest.mark.parametrize("user_in_city", ["Pune", "Bengaluru"], indirect=True)
def test_indirect_parametrize_builds_data_in_a_fixture(user_in_city: NewUser) -> None:
    assert user_in_city.city in {"Pune", "Bengaluru"}


"""
 INTERVIEW
  Q  "How do you write data-driven tests in pytest?"
  A  "@pytest.mark.parametrize with readable ids, or pytest.param for a per-row id or mark, like
      marking one known-broken row xfail. Large data lives in JSON or CSV loaded at collection
      time, so adding a case means adding a row. If a row needs setup, like creating a user,
      I use indirect parametrization so the fixture builds and cleans it up."
  Follow-ups
   Q "parametrize vs fixture params=[...]?"
   A "parametrize belongs to one test. A parametrized fixture (params=[...]) multiplies EVERY
      test that uses it, e.g. run all API tests against two API versions."
  Mistake  Loops inside one test: the first failing row hides the rest, and the report shows
           one test instead of N.
"""
# endregion


# region 6. MARKERS, SELECTION, SKIP AND XFAIL
"""
================================================================================
 6. MARKERS, SELECTION, SKIP AND XFAIL
================================================================================
 HOOK  Markers are tags (TS: { tag: ['@smoke'] }). -m selects by marker, -k by name.

   pytest -m smoke                      only smoke
   pytest -m "api and not creates_data" boolean expressions
   pytest -k "search and not json"      substring match on test names / ids
```

```
   pytest --lf / --ff                   last-failed only / failed first
   pytest -x  /  --maxfail=3            stop at the first / third failure
   pytest --collect-only -q             list what WOULD run (great for checking a -m filter)
   pytest --durations=10                the 10 slowest tests
   pytest -s  /  -v  /  -q              show print() / verbose / quiet
   pytest path/test_x.py::TestC::test_y run exactly one test (node id)

 --strict-markers (in pytest.ini) turns a typo like @pytest.mark.smok into an ERROR instead of a
 silently unselected test. Markers are registered in pytest.ini.
"""


@pytest.mark.smoke
def test_products_list_matches_schema(products_api: ProductsApi) -> None:
    body = expect_schema(ProductsList, products_api.list().body, "ProductsList")
    assert body.responseCode == 200 and body.products


@pytest.mark.skipif(os.name != "nt", reason="Windows-only path handling")
def test_skipif_with_a_condition() -> None:
    assert Path("C:/temp").drive == "C:"


@pytest.mark.xfail(reason="KNOWN DEFECT: API returns HTTP 200 for 'not found'", strict=True)
def test_known_defect_api_returns_200_for_errors(auth_api: AuthApi) -> None:
    result = auth_api.verify_login("nobody@example.com", "wrong")
    assert result.http_status == 404  # fails today → reported as XFAIL, the run stays green
    # strict=True (also xfail_strict in pytest.ini): when the defect is FIXED this test XPASSes
    # and the run FAILS, which tells you to remove the marker. Without strict it hides forever.


def test_skip_from_inside_a_test(env_config: EnvironmentConfig) -> None:
    if env_config.name == "local":
        pytest.skip("needs the shared QA server")  # imperative skip, decided at runtime
    assert env_config.base_url.startswith("http")


"""
 INTERVIEW
  Q  "skip vs xfail?"
  A  "skip means don't run it: wrong platform, missing dependency, not applicable to this env.
      xfail means run it and expect a failure: a known, tracked defect. I make xfail strict so a
      fix shows up as XPASS and fails the run, and the marker gets removed with a link to the
      ticket."
  Q  "How do you organise smoke vs regression?"
  A  "Registered markers with --strict-markers. CI runs -m smoke on every commit and the full
      suite nightly. A collection hook can also auto-mark by folder (section 7)."
  Mistake  Unregistered markers: a typo silently deselects the test in CI.
"""
# endregion


# region 7. HOOKS, PLUGINS AND CONFIGURATION
"""
═══════════════════════════════════════════════════════════════════════════════
 7. HOOKS, PLUGINS AND CONFIGURATION
═══════════════════════════════════════════════════════════════════════════════
 HOOK  conftest.py hooks change pytest itself; a plugin is a packaged conftest (pip install).

 HOOKS USED IN conftest.py
   pytest_addoption(parser)                  adds --env (TS: TEST_ENV)
   pytest_collection_modifyitems(config, items)
                                             after collection: skip creates_data on prod,
                                             auto-add the `api` marker by file name
 OTHER HOOKS YOU SHOULD KNOW
   pytest_configure(config)                  register markers / set globals at startup
   pytest_runtest_makereport(item, call)     a hookwrapper that sees each test's result → the
                                             standard way to screenshot or attach logs ON FAILURE
   pytest_sessionfinish(session, exitstatus) run-level summary, notify Slack, etc.

 CONFIG FILE PRECEDENCE: pytest.ini > pyproject.toml [tool.pytest.ini_options] > tox.ini/setup.cfg
   addopts = -ra --strict-markers     applied to every run
   testpaths = tests                  where to look when no path is given
   markers = ...                      registration for --strict-markers
   xfail_strict = true

 DISCOVERY RULES (defaults)
   files    test_*.py or *_test.py
   classes  Test* WITHOUT an __init__ method
   funcs    test_* (module level or inside Test* classes)
```

```
 THE PLUGINS A LEAD IS EXPECTED TO KNOW
   pytest-xdist          parallel: -n auto, --dist loadscope / loadfile / loadgroup
   pytest-playwright     browser fixtures: page, context, browser; --browser, --headed, --tracing
   pytest-rerunfailures  --reruns 2 / @pytest.mark.flaky(reruns=2), reported as RERUN
   allure-pytest         --alluredir=allure-results; allure.step, @allure.feature
   pytest-html           --html=report.html --self-contained-html
   pytest-cov            --cov=api --cov-report=xml   (coverage of FRAMEWORK code, unit tests)
   pytest-timeout        --timeout=60, kills hung tests
   pytest-mock           `mocker` fixture around unittest.mock
   pytest-bdd            Gherkin .feature files with step fixtures
   pytest-asyncio        async def tests
"""


def test_cli_option_reaches_fixtures(pytestconfig: pytest.Config, env_config: EnvironmentConfig) -> None:
    assert env_config.name == pytestconfig.getoption("--env")  # --env → hook → fixture


@pytest.mark.creates_data
def test_collection_hook_auto_marks_api_files(request: pytest.FixtureRequest) -> None:
    # This file's name doesn't contain "api", so the hook did NOT add the marker…
    assert request.node.get_closest_marker("api") is None
    # …but it is marked creates_data, so `pytest --env prod` reports it as SKIPPED.
    assert request.node.get_closest_marker("creates_data") is not None


"""
 EXAMPLE (not active here): screenshot on failure with pytest-playwright, in conftest.py

   @pytest.hookimpl(hookwrapper=True)
   def pytest_runtest_makereport(item, call):
       outcome = yield                                  # let pytest build the report
       report = outcome.get_result()
       if report.when == "call" and report.failed and "page" in item.funcargs:
           page = item.funcargs["page"]
           allure.attach(page.screenshot(), name="failure", attachment_type=allure.attachment_type.PNG)

 INTERVIEW
  Q  "How do you add a custom command-line option, like --env?"
  A  "pytest_addoption in the root conftest.py, read it with request.config.getoption or the
      pytestconfig fixture, and expose it through a session fixture so tests never read options
      directly. I default it from an environment variable so CI can set TEST_ENV."
  Q  "How do you take a screenshot only when a test fails?"
  A  "A pytest_runtest_makereport hookwrapper: after the call phase, if the report failed and the
      test used the page fixture, attach page.screenshot() to Allure. pytest-playwright also
      has --screenshot only-on-failure built in."
  Q  "What are pytest's exit codes?"
  A  "0 all passed, 1 some failed, 2 interrupted, 3 internal error, 4 usage error, 5 no tests
      collected. Exit 5 matters in CI: a wrong -m filter should fail the build, not pass."
"""
# endregion


# region 8. RUNTIME VALIDATION WITH PYDANTIC
"""
═══════════════════════════════════════════════════════════════════════════════
 8. RUNTIME VALIDATION WITH PYDANTIC (= zod)
═══════════════════════════════════════════════════════════════════════════════
 HOOK  One model → the runtime check AND the type hints. (zod: one schema → check + z.infer)

   zod                                pydantic v2
   z.object({ id: z.number() })       class X(BaseModel): id: int
   schema.safeParse(body)             X.model_validate(body)  (raises ValidationError)
   z.infer<typeof X>                  X itself IS the type
   .min(1) / .email() / .refine()     Field(min_length=1) / EmailStr / @field_validator
   z.array(X)                         list[X]
   .strict() (no extra keys)          model_config = ConfigDict(extra="forbid")

 framework.expect_schema(model, body) is the Python expectSchema<T>: generic over the model,
 returns a typed instance, and fails with every broken field path, e.g. "products.0.price".
"""


def test_valid_body_becomes_a_typed_object(products_api: ProductsApi) -> None:
    body = expect_schema(ProductsList, products_api.list().body)
    assert isinstance(body.products[0], Product)  # typed objects, not raw dicts
    assert body.products[0].name == "Blue Top"  # autocomplete works on body.products[0].


def test_invalid_body_fails_with_the_exact_field_path() -> None:
    broken = {"responseCode": 200, "products": [{"id": 1, "name": "Blue Top"}]}  # price missing
```

```
    with pytest.raises(AssertionError, match=r"products\.0\.price: Field required"):
        expect_schema(ProductsList, broken)


class CreateUserRequest(BaseModel):
    name: str = Field(min_length=1)
    email: str
    age: int = Field(ge=18)

    @field_validator("email")
    @classmethod
    def email_has_at(cls, value: str) -> str:
        if "@" not in value:
            raise ValueError("must contain @")
        return value.lower()  # validators can also NORMALISE


def test_pydantic_constraints_and_validators() -> None:
    assert CreateUserRequest(name="QA", email="QA@Example.com", age=30).email == "qa@example.com"
    assert CreateUserRequest.model_validate({"name": "QA", "email": "a@b.c", "age": "30"}).age == 30
    # ↑ lax mode COERCES "30" → 30. For API contract tests where "30" must be a defect, use
    #   strict mode: model_config = ConfigDict(strict=True).
    with pytest.raises(ValueError, match="must contain @"):
        CreateUserRequest(name="QA", email="not-an-email", age=30)


"""
 INTERVIEW
  Q  "How do you validate API responses in Python?"
  A  "pydantic models per response. model_validate turns the JSON into typed objects or raises
      with every failing field path, so a contract break is one clear failure instead of a
      KeyError three lines later. The same models type the rest of the test."
  Follow-ups
   Q "pydantic vs jsonschema?"
   A "jsonschema validates against a JSON Schema document, good when the API publishes one.
      pydantic gives typed Python objects too; and model_json_schema() can export a schema."
  Mistake  Asserting `response.json()["products"][0]["price"]` everywhere: no contract check,
           and a renamed field fails with a confusing KeyError.
"""
# endregion


# region 9. DESIGN PATTERNS IN PYTHON
"""
═══════════════════════════════════════════════════════════════════════════════
 9. DESIGN PATTERNS IN PYTHON (the same 6 as the TypeScript notes)
═══════════════════════════════════════════════════════════════════════════════
   Page Object + Component  "tests say WHAT"      pages/login_page.py (below)
   Builder / Factory        "order the sandwich"  framework.build_user, UserBuilder
   Facade                   "one button"          framework.ProductsApi.search
   Strategy                 "swap the tool"       framework.ENVIRONMENTS + --env
   Singleton                "one key"             a module is imported once; @functools.cache
   Adapter                  "travel plug"         framework.expect_api_code (HTTP 200 quirk)

 PAGE OBJECT with Playwright for Python (sync API). Not executed here (needs a browser):

   from playwright.sync_api import Page, expect

   class LoginPage:
       def __init__(self, page: Page) -> None:
           self.page = page
           panel = page.locator("#loginPanel")                      # scope to the panel
           self.username = panel.locator('input[name="username"]')  # selectors live HERE only
           self.password = panel.locator('input[name="password"]')
           self.login_button = panel.get_by_role("button", name="Log In")

       def open(self) -> "LoginPage":
           self.page.goto("/parabank/index.htm")                    # relative to --base-url
           return self

       def login(self, username: str, password: str) -> None:       # a BUSINESS action
           self.username.fill(username)
           self.password.fill(password)
           self.login_button.click()

   def test_valid_login(page: Page, credentials) -> None:            # page = pytest-playwright
       LoginPage(page).open().login(credentials.username, credentials.password)
       expect(page.get_by_role("heading", name="Accounts Overview")).to_be_visible()
"""
```

```python
def test_builder_defaults_and_overrides() -> None:
    user = build_user(city="Bengaluru")
    assert user.city == "Bengaluru" and user.country == "India"  # override one, keep the rest
    assert build_user().email != build_user().email  # unique per call (parallel-safe)
    with pytest.raises(AttributeError):
        user.city = "Delhi"  # type: ignore[misc]  frozen dataclass: test data can't be mutated


def test_fluent_builder_returns_fresh_objects() -> None:
    builder = UserBuilder().with_role("admin")
    first, second = builder.build(), builder.build()
    assert first["role"] == "admin"
    assert first is not second  # reusing the builder never shares one object


@functools.cache  # SINGLETON via cache: the body runs once, later calls return the same object
def settings() -> dict[str, str]:
    return {"base_url": os.getenv("BASE_URL", "http://127.0.0.1:8080")}


def test_cached_function_is_a_singleton() -> None:
    assert settings() is settings()
    # (!) once per PROCESS = once per xdist worker. Keep it read-only.


def test_adapter_hides_the_always_200_quirk(auth_api: AuthApi) -> None:
    result = auth_api.verify_login("nobody@example.com", "wrong")
    assert result.http_status == 200  # the awkward truth…
    expect_api_code(result, 404, "User not found!")  # …the readable check specs use


"""
 INTERVIEW
  Q  "Which design patterns do you use in a pytest framework?"
  A  "Page and component objects for selectors; a builder for unique, valid test data; service
      classes as a facade over requests; an environment strategy chosen by --env; config as a
      module-level or cached singleton; an adapter for an API that always returns 200. And
      fixtures for dependency injection with guaranteed cleanup."
  Q  "Should page objects contain assertions?"
  A  "Not business assertions. They perform actions and expose locators; the test asserts with
      expect(). Playwright's expect already auto-waits, so no sleeps in page objects either."
  Mistake  A BasePage with 60 helpers that every page inherits; prefer small components (HAS-A).
"""
# endregion


# region 10. PLAYWRIGHT FOR PYTHON (pytest-playwright)
"""
═══════════════════════════════════════════════════════════════════════════════
 10. PLAYWRIGHT FOR PYTHON (pytest-playwright)
═══════════════════════════════════════════════════════════════════════════════
 HOOK  Same engine, same locators and auto-waiting; the runner is pytest instead of
       Playwright Test, so fixtures/config move to conftest.py and CLI flags.

 SETUP
   pip install pytest-playwright
   playwright install --with-deps chromium

 FIXTURES IT PROVIDES        TS equivalent
   page                      { page }            function scope, new context per test
   context                   { context }
   browser                   { browser }         session scope
   browser_name              project name        parametrized by --browser
   base_url                  use.baseURL         from --base-url (pytest-base-url)
   browser_context_args      use: { ... }        OVERRIDE it to set viewport, storage_state, locale

 CLI ≈ playwright.config.ts
   pytest --browser chromium --browser firefox     run on 2 browsers (like 2 projects)
   pytest --headed --slowmo 200                    watch it
   pytest --tracing retain-on-failure --video retain-on-failure --screenshot only-on-failure
   pytest --base-url https://parabank.parasoft.com
   pytest -n 4                                     parallel via pytest-xdist (no built-in workers)

 SAVED LOGIN SESSION (TS: storageState, the 24 → 1 logins fix), in tests/ui/conftest.py

   @pytest.fixture(scope="session")
   def auth_state(browser, base_url, tmp_path_factory) -> str:
       path = tmp_path_factory.mktemp("auth") / "state.json"
       context = browser.new_context(base_url=base_url)
       LoginPage(context.new_page()).open().login(USERNAME, PASSWORD)   # log in ONCE
       context.storage_state(path=str(path))                            # save cookies
```

```
        context.close()
        return str(path)

    @pytest.fixture
    def browser_context_args(browser_context_args, auth_state):       # override built-in
        return {**browser_context_args, "storage_state": auth_state}  # every page starts logged in

 KEY DIFFERENCES FROM PLAYWRIGHT TEST (TypeScript)
   • No built-in parallelism or retries → pytest-xdist and pytest-rerunfailures.
   • No HTML reporter / trace viewer integration by default → --tracing + `playwright show-trace`,
     allure-pytest or pytest-html.
   • Sync API by default: no await, so the "missing await" flakiness class disappears.
   • expect() is imported: from playwright.sync_api import expect. Same auto-retrying assertions.
   • API testing: page.request / playwright.request.new_context(), or plain requests/httpx.

 INTERVIEW
  Q  "How would you move a Playwright TS suite to Python?"
  A  "Page objects port almost line by line because locators and expect are the same API. The
      config moves: projects become --browser flags, use.baseURL becomes --base-url, worker
      fixtures become session fixtures, test.extend becomes conftest.py, retries come from
      pytest-rerunfailures and parallelism from pytest-xdist. I'd port the fixtures and one page
      first, run them side by side, then move feature by feature."
"""
# endregion


# region 11. PARALLEL RUNS, FLAKINESS AND CI
r"""
═══════════════════════════════════════════════════════════════════════════════
 11. PARALLEL RUNS, FLAKINESS AND CI
═══════════════════════════════════════════════════════════════════════════════
 HOOK  xdist workers are separate PROCESSES: no shared memory, session fixtures run per worker.

 pytest-xdist
   pytest -n auto                one worker per CPU
   --dist load (default)         any test to any free worker
   --dist loadscope              keep a module/class on one worker (share module fixtures)
   --dist loadfile               keep a file on one worker
   --dist loadgroup              @pytest.mark.xdist_group("bank-db") keeps a group together
   worker_id fixture             "gw0", "gw1"… ("master" without xdist) → per-worker data names

 MAKING TESTS PARALLEL-SAFE (the same rules that fixed the TS suite)
   • Unique data per test: builders with uuid (framework.unique_id) → no email clashes.
   • No order dependency: every test creates what it needs (install pytest-randomly: it shuffles test order on every run).
   • Shared servers rate-limit: log in once per worker (session fixture + storage_state) and
     cap workers per environment (ENVIRONMENTS[...].workers).
   • Never mutate session-scoped objects.

 FLAKINESS TRIAGE
   1. Reproduce:  pytest --count 20 -x (pytest-repeat) and shuffled order (pytest-randomly)
   2. Look for:   time.sleep instead of expect auto-wait · shared data · order dependency ·
                  rate limits · missing cleanup · environment slowness
   3. Retries:    --reruns 2 ONLY with reruns visible in the report, and a ticket per flaky test

 JENKINS / GITHUB ACTIONS COMMAND
   python -m pytest -m smoke -n auto --env qa \
          --junitxml=junit-results/pytest.xml \      ← Jenkins `junit` step reads this
          --alluredir=allure-results \               ← Allure Jenkins plugin
          --reruns 1 --timeout 120
   Use `python -m pytest` (not bare `pytest`): it uses the venv's interpreter and adds the cwd
   to sys.path. The exit code decides the build status (5 = no tests collected → fail).
"""


@pytest.mark.creates_data
def test_unique_data_makes_parallel_runs_safe(make_user) -> None:
    emails = {make_user().email for _ in range(20)}
    assert len(emails) == 20  # no clashes even when 20 users are created at once


def test_no_users_left_behind(ae_server: FakeAutomationExerciseServer) -> None:
    """Runs after the data tests above (file order). Proves the yield/factory teardowns worked.
    (!) Deliberately order-dependent to PROVE cleanup; real tests must never rely on order."""
    assert ae_server.users == {}


"""
 INTERVIEW
  Q  "A test passes locally but fails intermittently in CI. What do you check?"
  A  "First whether it fails alone or only in parallel: if only in parallel, it's shared data or
      order dependency, so I check that every test builds unique data and cleans up. Then waits:
```

```
      any time.sleep or manual polling instead of expect's auto-wait. Then the environment: rate
      limits or slow responses on the shared server. I reproduce with repeated and randomised
      runs, fix the cause, and only then consider a visible rerun."
  Q  "Is a session fixture shared between xdist workers?"
  A  "No. Each worker is its own process and gets its own session fixtures. That's fine for an
      HTTP session or a login; for something that must happen once, use a file lock in the
      shared temp dir."
"""
# endregion


# region 12. ONE TEST, END TO END
"""
================================================================================
 12. ONE TEST, END TO END   (pytest notes/python --setup-show -k end_to_end)
================================================================================
"""


@pytest.mark.smoke
@pytest.mark.creates_data
def test_end_to_end_valid_login_returns_user_exists(auth_api: AuthApi, registered_user: NewUser) -> None:
    result = auth_api.verify_login(registered_user.email, registered_user.password)
    expect_api_code(result, 200, "User exists!")


"""
 WHAT HAPPENS, STEP BY STEP
  1. Collection   pytest imports this file, finds test_* functions, reads markers (smoke,
                  creates_data); pytest.ini's --strict-markers checks they're registered.
  2. Hook         conftest.pytest_collection_modifyitems: on --env prod this test gets a skip.
  3. Resolve      parameters auth_api, registered_user → fixture graph:
                  auth_api ← api_client ← ae_server · registered_user ← account_api ← api_client
                  (+ the autouse _count_requests_per_test).
  4. Session      ae_server built once for the whole run (already exists for later tests).
  5. Setup        api_client, auth_api, account_api built; registered_user: build_user() (unique
                  email) → AccountApi.create → ApiClient.send → expect_api_code(201) proves it.
  6. Call         the test body: AuthApi.verify_login → ApiClient.send (the one "HTTP" place).
  7. Assert       expect_api_code (adapter): HTTP 200 + pydantic ApiMessage + code 200 + message.
  8. Teardown     after `yield` in registered_user: delete the account (on pass AND fail);
                  then the other fixtures in reverse order; autouse fixture records api_requests.
  9. Report       pass/fail + user_properties into --junitxml for Jenkins.

 60-SECOND ANSWER: "Walk me through what happens when one of your tests runs."
  "pytest collects the test and its markers, and a collection hook applies environment rules,
   like skipping data-changing tests on prod. It then resolves the fixtures the test names: the
   auth service, and a registered user whose fixture builds unique data and creates a real
   account through the API service and client. The test body is one business call and one
   assertion helper that validates the body with pydantic. After the test, even on failure,
   the fixture deletes the account, and the result goes to JUnit XML and Allure."
"""
# endregion


# region 13. TYPESCRIPT → PYTHON CHEAT SHEET
"""
================================================================================
 13. TYPESCRIPT → PYTHON CHEAT SHEET
================================================================================
  Playwright Test (TS)                          pytest (+ plugins)
  ──────────────────────────────────────────── ──────────────────────────────────────────────
  test('name', async ({ page }) => {})          def test_name(page): ...
  test.describe('group', () => {})              class TestGroup:   (or one module per group)
  test.extend<{ x }>({ x: async ({}, use) })    @pytest.fixture def x(): ... yield value
  await use(value) + code after                yield value + code after
  { scope: 'worker' }                           scope="session"  (per xdist worker)
  { auto: true }                                autouse=True
  test.beforeEach / afterEach                   autouse function fixture with yield
  test.beforeAll / afterAll                     module/class-scoped fixture
  { tag: ['@smoke'] } + --grep @smoke           @pytest.mark.smoke + -m smoke
  --grep "login"                                -k login
  test.skip(cond, 'why') / test.fixme           pytest.skip("why") / @pytest.mark.skip
  test.fail()                                   @pytest.mark.xfail(strict=True)
  for (const row of rows) test(...)             @pytest.mark.parametrize
  test.step('Log in', async () => {})           with allure.step("Log in"):
  retries: 2                                    --reruns 2 (pytest-rerunfailures)
  workers: 4 / fullyParallel                    -n 4 (pytest-xdist)
  projects: [chromium, firefox]                 --browser chromium --browser firefox
  use: { baseURL, storageState }                --base-url + browser_context_args fixture
  playwright.config.ts                          pytest.ini / pyproject.toml + conftest.py
  process.env.TEST_ENV                          os.getenv("TEST_ENV") / pytest_addoption --env
```

```
  zod schema + z.infer                    pydantic BaseModel
  interface / type                        Protocol / TypedDict / dataclass
  readonly, as const                      @dataclass(frozen=True), tuple, Final
  { ...a, ...b }                          {**a, **b}  /  dataclasses.replace(obj, **b)
  structuredClone(x)                      copy.deepcopy(x)
  arr.map / filter                        [f(x) for x in arr if cond]
  async/await (required)                  sync API by default (no await)
  try / finally                           try / finally, or a `with` context manager
  npm ci / package-lock.json              pip install -r requirements.txt (or uv / poetry lock)
"""
# endregion


# region TEST YOURSELF
"""
═══════════════════════════════════════════════════════════════════════════════
 TEST YOURSELF (answer out loud, then check the section)
═══════════════════════════════════════════════════════════════════════════════
  1. Why is `def f(items=[])` a bug, and what are the two fixes?                (1.1)
  2. When do you use `is` instead of `==`?                                      (1.2)
  3. A test changes a nested field and another test's data changes too. Why?     (1.3)
  4. Write a retry decorator. Why functools.wraps? What should it NOT catch?    (1.6)
  5. What does `yield` do in a generator, and what does it do in a fixture?  (1.4, 4)
  6. Type hints are not checked at runtime: so what do you do with API responses?  (2, 8)
  7. What's a Protocol and when is it useful in tests?                             (2)
  8. Name the 5 fixture scopes. Which one maps to Playwright's worker scope?       (4)
  9. Setup before `yield` raises: does the teardown run? How do you protect cleanup?  (4)
 10. Fixtures vs setup_method? Fixtures vs beforeEach?                             (4)
 11. A test needs 3 users. How, without 3 fixtures?                                (4)
 12. How does a fixture in a sub-folder conftest.py override one in the root?   (3, 4)
 13. parametrize vs parametrized fixture vs indirect parametrize?                  (5)
 14. skip vs xfail. Why xfail strict=True?                                         (6)
 15. Why --strict-markers?                                                         (6)
 16. Add a --env option and make prod skip data-changing tests: which hooks?       (7)
 17. Screenshot only on failure: how?                                         (7, 10)
 18. Are session fixtures shared across xdist workers? Once-per-run setup?    (4, 11)
 19. Port a Playwright TS config (projects, baseURL, storageState, retries) to pytest.  (10, 13)
 20. Flaky in CI, green locally: your checklist?                                  (11)

  BONUS  Why don't plain asserts in framework.py show pytest's rich diff? (pytest rewrites
         asserts only in test files and conftest.py; call pytest.register_assert_rewrite("api")
         for helper packages, or write explicit messages as framework.py does. Also: `python -O`
         strips asserts entirely, so never use assert for non-test validation.)
"""
# endregion
```

Appendix A: conftest.py (fixtures + hooks)

```
"""
LAYER 2: WIRING. conftest.py is pytest's fixtures/apiTest.ts + fixtures/baseTest.ts.

  - pytest finds conftest.py automatically (never import it). Fixtures here are visible to every
    test in this folder and below; a conftest.py in a sub-folder can add or OVERRIDE fixtures.
  - Hooks here (pytest_addoption, pytest_collection_modifyitems, ...) customise pytest itself.

Guide: test_python_pytest_essentials.py, section 4 (fixtures) and section 7 (hooks).
"""

from __future__ import annotations

import os
from collections.abc import Callable, Iterator

import pytest

from framework import (
    AccountApi,
    ApiClient,
    AuthApi,
    EnvironmentConfig,
    FakeAutomationExerciseServer,
    NewUser,
    ProductsApi,
    build_user,
    expect_api_code,
    get_environment,
)

# ── HOOKS ──────────────────────────────────────────────────────────────────────


def pytest_addoption(parser: pytest.Parser) -> None:
    """Adds `pytest --env prod`. TS equivalent: TEST_ENV=prod npx playwright test."""
    parser.addoption(
        "--env",
        default=os.getenv("TEST_ENV", "qa"),  # CLI flag wins, then the env var, then a default
        choices=["local", "qa", "staging", "prod"],
        help="Target environment (strategy) for the run",
    )


def pytest_collection_modifyitems(config: pytest.Config, items: list[pytest.Item]) -> None:
    """Runs once after collection: a central place for run-wide rules.

    Rule 1 (safety): tests marked creates_data are SKIPPED on an env that forbids data changes.
    Rule 2 (convenience): any test whose file name contains "api" gets the api marker for free.
    """
    env = get_environment(config.getoption("--env"))
    skip_prod = pytest.mark.skip(reason=f"{env.name} does not allow data changes")
    for item in items:
        if not env.allow_data_changes and item.get_closest_marker("creates_data"):
            item.add_marker(skip_prod)
        if "api" in item.path.name:
            item.add_marker(pytest.mark.api)


# ── FIXTURES ───────────────────────────────────────────────────────────────────


@pytest.fixture(scope="session")
def env_config(pytestconfig: pytest.Config) -> EnvironmentConfig:
    """Session scope = once per run (per xdist WORKER when run in parallel)."""
    return get_environment(pytestconfig.getoption("--env"))


@pytest.fixture(scope="session")
def ae_server() -> Iterator[FakeAutomationExerciseServer]:
    """Expensive shared resource → session scope. Real life: a requests.Session / APIRequestContext."""
    server = FakeAutomationExerciseServer()  # SETUP (once)
    yield server  # ← every test that needs it runs here
    server.users.clear()  # TEARDOWN (once, after the last test)


@pytest.fixture
def api_client(ae_server: FakeAutomationExerciseServer) -> ApiClient:
    return ApiClient(ae_server)  # function scope (default): fresh per test, cheap
```

```python
@pytest.fixture
def products_api(api_client: ApiClient) -> ProductsApi:
    return ProductsApi(api_client)  # composition: service HAS a client


@pytest.fixture
def account_api(api_client: ApiClient) -> AccountApi:
    return AccountApi(api_client)


@pytest.fixture
def auth_api(api_client: ApiClient) -> AuthApi:
    return AuthApi(api_client)


@pytest.fixture
def registered_user(account_api: AccountApi) -> Iterator[NewUser]:
    """TS registeredUser: create → yield to the test → ALWAYS delete (even if the test failed)."""
    user = build_user()  # 1. unique, valid data
    expect_api_code(account_api.create(user), 201, "User created!")  # 2. real account, proven
    yield user  # 3. ← THE TEST RUNS HERE
    account_api.delete(user.email, user.password)  # 4. teardown, runs on pass AND fail


@pytest.fixture
def make_user(account_api: AccountApi) -> Iterator[Callable[..., NewUser]]:
    """FACTORY-AS-FIXTURE: when one test needs SEVERAL users. Cleans up every one it made."""
    created: list[NewUser] = []

    def _make(**overrides: str) -> NewUser:
        user = build_user(**overrides)
        expect_api_code(account_api.create(user), 201)
        created.append(user)
        return user

    yield _make
    for user in created:  # a plain for loop: each delete finishes before the next
        account_api.delete(user.email, user.password)


@pytest.fixture(autouse=True)
def _count_requests_per_test(ae_server: FakeAutomationExerciseServer, request: pytest.FixtureRequest) -> Iterator[None]:
    """AUTOUSE: runs for EVERY test without being requested (TS: { auto: true }).
    Here it records how many API calls each test made, on the test node, for reporting."""
    before = ae_server.request_count
    yield
    request.node.user_properties.append(("api_requests", ae_server.request_count - before))  # → junit xml
```

Appendix B: framework.py (layers 3 and 4)

```
"""
A tiny, OFFLINE Python mirror of the TypeScript framework's lower layers (layers 3 and 4).

The prep guide (test_python_pytest_essentials.py) and conftest.py use it so every example
really runs under pytest, without a browser or network:

    test_python_pytest_essentials.py   LAYER 1  WHAT     (specs: business expectations only)
    conftest.py                        LAYER 2  WIRING   (fixtures = dependency injection)
    framework.py  (this file)          LAYER 3  HOW      (ApiClient, services, assertions)
                                       LAYER 4  WHERE    (environments, schemas, test data)

TypeScript original → Python equivalent used here
    api/automation-exercise/ApiClient.ts           → ApiClient
    api/automation-exercise/services/*Api.ts       → ProductsApi, AccountApi, AuthApi
    api/automation-exercise/schemas.ts (zod)       → pydantic models
    api/automation-exercise/assertions.ts          → expect_schema, expect_api_code
    api/automation-exercise/test-data/userBuilder  → build_user, UserBuilder
    config/environments.ts                         → ENVIRONMENTS, get_environment
"""

from __future__ import annotations

import itertools
import uuid
from dataclasses import asdict, dataclass, field, replace
from typing import Any, Literal, TypeVar

from pydantic import BaseModel, ValidationError

# ── LAYER 4: WHERE ──────────────────────────────────────────────────────────
# STRATEGY pattern: one config object per environment; TEST_ENV / --env picks one at runtime.

EnvironmentName = Literal["local", "qa", "staging", "prod"]
PARABANK_URL = "https://parabank.parasoft.com/parabank"


@dataclass(frozen=True)  # frozen = read-only, like `as const`/readonly in TS → safe to share
class EnvironmentConfig:
    name: EnvironmentName
    base_url: str
    allow_data_changes: bool  # safety rule: prod must never create or delete data
    workers: int


ENVIRONMENTS: dict[str, EnvironmentConfig] = {
    "local": EnvironmentConfig("local", "http://127.0.0.1:8080", True, 4),
    "qa": EnvironmentConfig("qa", PARABANK_URL, True, 2),
    "staging": EnvironmentConfig("staging", PARABANK_URL, True, 2),
    "prod": EnvironmentConfig("prod", PARABANK_URL, False, 2),
}


def get_environment(name: str) -> EnvironmentConfig:
    """Fail fast with a helpful message instead of a KeyError deep inside a test."""
    try:
        return ENVIRONMENTS[name]
    except KeyError:
        raise ValueError(f"Unknown environment {name!r}. Use one of: {', '.join(ENVIRONMENTS)}") from None


# ── LAYER 4: SCHEMAS (pydantic = zod) ───────────────────────────────────────


class ApiMessage(BaseModel):
    responseCode: int
    message: str


class Product(BaseModel):
    id: int
    name: str
    price: str  # the real API returns "Rs. 500" (a string), so the schema says str


class ProductsList(BaseModel):
    responseCode: int
    products: list[Product]
```

```python
M = TypeVar("M", bound=BaseModel)  # generic: "any pydantic model" (TS: <T> with ZodType<T>)


def expect_schema(model: type[M], body: Any, schema_name: str | None = None) -> M:
    """TS expectSchema<T>: validate a raw body and return it typed as exactly `model`."""
    try:
        return model.model_validate(body)
    except ValidationError as error:
        details = "\n".join(
            f"  • {'.'.join(str(p) for p in issue['loc']) or '(root)'}: {issue['msg']}"
            for issue in error.errors()
        )
        raise AssertionError(f"Response does not match {schema_name or model.__name__} schema:\n{details}") from None


# ── LAYER 4: TEST DATA (Builder) ──────────────────────────────────────────

_counter = itertools.count(1)


def unique_id() -> str:
    """Unique across parallel xdist workers too: uuid part differs per process."""
    return f"{next(_counter)}{uuid.uuid4().hex[:6]}"


@dataclass(frozen=True)
class NewUser:
    name: str
    email: str
    password: str
    title: str = "Mr"
    city: str = "Pune"
    country: str = "India"


def build_user(**overrides: Any) -> NewUser:
    """Valid by default; a test overrides only what it is about. TS: buildUser(overrides)."""
    uid = unique_id()
    defaults = NewUser(name=f"QA User {uid}", email=f"qa.api.{uid}@example.com", password=f"Pw_{uid}!")
    return replace(defaults, **overrides)  # replace() = { ...defaults, ...overrides }, still frozen


@dataclass
class UserBuilder:
    """FLUENT builder (the class version interviewers ask for)."""

    _role: Literal["admin", "user"] = "user"
    _overrides: dict[str, Any] = field(default_factory=dict)  # NOT `= {}`: mutable default trap

    def with_role(self, role: Literal["admin", "user"]) -> UserBuilder:
        self._role = role
        return self  # returning self allows chaining

    def with_email(self, email: str) -> UserBuilder:
        self._overrides["email"] = email
        return self

    def build(self) -> dict[str, Any]:
        return {**asdict(build_user(**self._overrides)), "role": self._role}  # a fresh dict each time


# ── LAYER 3: HOW (an in-memory fake server + client + services) ──────────


class FakeAutomationExerciseServer:
    """Stands in for automationexercise.com. Quirk kept on purpose: ALWAYS HTTP 200."""

    def __init__(self) -> None:
        self.users: dict[str, NewUser] = {}
        self.products = [
            {"id": 1, "name": "Blue Top", "price": "Rs. 500"},
            {"id": 2, "name": "Men Tshirt", "price": "Rs. 400"},
            {"id": 3, "name": "Sleeveless Dress", "price": "Rs. 1000"},
        ]
        self.request_count = 0

    def handle(self, method: str, path: str, form: dict[str, str] | None) -> tuple[int, dict[str, Any]]:
        self.request_count += 1
        form = form or {}
        if (method, path) == ("GET", "productsList"):
            return 200, {"responseCode": 200, "products": self.products}
```

```
        if (method, path) == ("POST", "searchProduct"):
            if "search_product" not in form:
                return 200, {"responseCode": 400, "message": "Bad request, search_product parameter is missing in POST request."
                    }
            term = form["search_product"].lower()
            return 200, {"responseCode": 200, "products": [p for p in self.products if term in p["name"].lower()]}
        if (method, path) == ("POST", "createAccount"):
            if form["email"] in self.users:
                return 200, {"responseCode": 400, "message": "Email already exists!"}
            self.users[form["email"]] = NewUser(**{k: form[k] for k in ("name", "email", "password")})
            return 200, {"responseCode": 201, "message": "User created!"}
        if (method, path) == ("DELETE", "deleteAccount"):
            if self.users.pop(form["email"], None) is None:
                return 200, {"responseCode": 404, "message": "Account not found!"}
            return 200, {"responseCode": 200, "message": "Account deleted!"}
        if (method, path) == ("POST", "verifyLogin"):
            user = self.users.get(form.get("email", ""))
            if user and user.password == form.get("password"):
                return 200, {"responseCode": 200, "message": "User exists!"}
            return 200, {"responseCode": 404, "message": "User not found!"}
        return 200, {"responseCode": 405, "message": "This request method is not supported."}


@dataclass(frozen=True)
class ApiResult:
    http_status: int
    body: Any


class ApiClient:
    """The ONE place that does 'HTTP'. With real HTTP this would wrap a requests.Session."""

    def __init__(self, server: FakeAutomationExerciseServer) -> None:
        self._server = server

    def send(self, method: str, path: str, form: dict[str, str] | None = None) -> ApiResult:
        status, body = self._server.handle(method, path, form)
        return ApiResult(status, body)


class ProductsApi:  # FACADE: tests say "search", the service knows method, path, form encoding
    def __init__(self, client: ApiClient) -> None:
        self.client = client

    def list(self) -> ApiResult:
        return self.client.send("GET", "productsList")

    def search(self, term: str | None) -> ApiResult:
        form = None if term is None else {"search_product": term}  # None → omit the parameter
        return self.client.send("POST", "searchProduct", form)


class AccountApi:
    def __init__(self, client: ApiClient) -> None:
        self.client = client

    def create(self, user: NewUser) -> ApiResult:
        return self.client.send("POST", "createAccount", asdict(user))

    def delete(self, email: str, password: str) -> ApiResult:
        return self.client.send("DELETE", "deleteAccount", {"email": email, "password": password})


class AuthApi:
    def __init__(self, client: ApiClient) -> None:
        self.client = client

    def verify_login(self, email: str, password: str) -> ApiResult:
        return self.client.send("POST", "verifyLogin", {"email": email, "password": password})


def expect_api_code(result: ApiResult, response_code: int, message: str | None = None) -> ApiMessage:
    """ADAPTER: this API always says HTTP 200, the real result is in the body."""
    assert result.http_status == 200, f"expected HTTP 200, got {result.http_status}"
    body = expect_schema(ApiMessage, result.body)
    assert body.responseCode == response_code, f"responseCode {body.responseCode} ({body.message!r}), expected {response_code}"
    if message is not None:
        assert body.message == message
    return body
```

Appendix C: pytest.ini, requirements.txt, search_cases.json

```
# ── pytest.ini ──────────────────────────────────────────────
# pytest config for the Python prep notes only (the real framework is TypeScript/Playwright).
# Run from the project root:  notes/python/.venv/Scripts/python -m pytest notes/python
[pytest]
testpaths = .
addopts = -ra --strict-markers
xfail_strict = true
markers =
    smoke: fast, critical-path checks (run on every commit)
    regression: the full suite (nightly)
    api: talks to an API layer
    creates_data: creates real data that a fixture must clean up

# ── requirements.txt ────────────────────────────────────────
# Only what the prep notes need to run offline. Install inside notes/python/.venv:
#   py -3.15 -m venv notes/python/.venv
#   notes/python/.venv/Scripts/python -m pip install -r notes/python/requirements.txt
pytest>=9.0
pydantic>=2.0

# ── search_cases.json ───────────────────────────────────────
[
  { "id": "exact-name", "term": "Blue Top", "expected_names": ["Blue Top"] },
  { "id": "case-insensitive", "term": "tshirt", "expected_names": ["Men Tshirt"] },
  { "id": "partial-match", "term": "dress", "expected_names": ["Sleeveless Dress"] },
  { "id": "no-results", "term": "spaceship", "expected_names": [] }
]
```